

# Evidence-Guided Repository-Level RTL Repair

Yuxin Du<sup>1</sup>, Juxin Niu<sup>1</sup>, Zhe Jiang<sup>2,3</sup>, and Nan Guan<sup>1</sup>

<sup>1</sup>City University of Hong Kong, Hong Kong, China

<sup>2</sup>Southeast University, China

<sup>3</sup>National Center of Technology Innovation for EDA, China

Email: yuxindu8-c@my.cityu.edu.hk, juxin.niu@my.cityu.edu.hk, 101013615@seu.edu.cn, nanguan@cityu.edu.hk

**Abstract**—Repository-level RTL repair must localize a failure that spans files, modules, and clock cycles, then propagate the fix consistently. Existing methods reason over source code, which reveals possible behaviors but not the failed execution, and cannot tell whether a local fix was propagated consistently. We therefore present an evidence-guided framework with three modules. Failure grounding converts a problem statement into a reproduced failing run and a failure anchor. Waveform-guided localization uses a localization toolbox to narrow the observed violation into a candidate mechanism and an evidence trail. Consistency-aware repair and validation then expand that seed into a coordinated patch and replay the same scenario to check that the violation disappears. We conducted experiments on HWE-Bench and achieved better performance than the baseline.

**Index Terms**—Repository-level RTL repair, waveform analysis, large language models.

## I. INTRODUCTION

Debugging RTL designs consumes much of the effort in chip development [1], [2]. Doing it by hand is tedious [3]. An engineer must trace the bug across files and modules, and every hypothesis needs another test run [4], [5]. Agents suit this task and are already applied to RTL debugging with promising results [6]–[10]. That progress, however, is confined to the module level [11]–[14]. At that level, the design unit and its testbench are supplied. Repository-level RTL bug repair instead takes a repository and a problem statement. The output is a patch that makes the repository pass the fail-to-pass tests. HWE-Bench [15] evaluates general-purpose agents on this setting, but no published method targets it.

RTL debuggers typically trace an observed failure back to its cause and then fix it [16]–[19]. This recipe gets much harder at repository scale. This task differs from small-scale repair in two ways. First, a repository contains many files and modules. The agent therefore needs a long reasoning chain to reach the root cause. That chain also crosses clock cycles. No specification or reference design is available to confirm a candidate cause [20], [21]. Correct localization becomes difficult [22], [23]. Second, repository-level bugs usually span several modules and files. Such a bug is solved only when every affected place is changed consistently. That requirement makes repair hard as well.

These two difficulties have a common cause: the agent lacks grounded evidence. Current debugging methods lean heavily on reasoning over source code. For small-scale RTL debugging, that ability is already strong enough. At repository

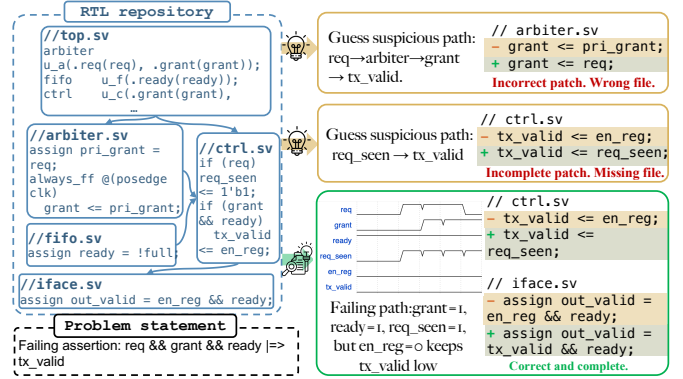


Fig. 1. Without cycle-level evidence, the agent tests hypotheses that are consistent with everything it can observe.

scale it stops working. Source code shows every path the design could take, but not the one this failing run took [24]–[26]. Hardware correctness is decided as the state evolves cycle by cycle, so many candidate causes fit everything the agent can read from the repository [27]. The agent can only test one at a time, and the loop does not converge. Figure 1 shows this from a real trajectory. Even after a local cause is found, the agent still lacks a reliable way to tell whether the same failing scenario has been preserved while the violation itself disappears. A patch can remove the observed symptom and still leave stale interfaces, widths, or state updates elsewhere. We therefore shift the approach from reasoning over source code alone to evidence-guided bug repair. The key evidence is the reproduced failing run and its waveform.

We present a framework for this task. It organizes the method into three modules. Failure grounding turns the problem statement into a reproduced failing run and a failure anchor that records the scenario, the observed violation, and the run to replay. Waveform-guided localization then uses a localization toolbox to bound the violation in time, trace active drivers, and lift raw values into design-level semantics, yielding a candidate mechanism plus an evidence trail. Consistency-aware repair and validation finally expand that local seed into a coordinated repository patch and replay the same scenario to check that the violation disappears without breaking the surrounding behavior. Because the framework reuses one observed execution throughout, it grounds both diagnosis and repair checking in what the design actually did

rather than in source-only reasoning.

We evaluate this framework on HWE-Bench, which poses real problem statements from large RTL repositories and judges each patch by fail-to-pass tests. It collects 417 tasks; 29 OpenTitan cases cannot be adapted to a runnable VCS environment in our setup, so every condition runs over the same 388 tasks. The framework resolves more of them than the baseline for every model we test.

## II. RELATED WORK

### A. Automated RTL Repair

Traditional hardware repair tools such as CirFix [11] and RTL-Repair [12] repair a given module against a supplied failing testbench, and other work debugs inside generation or verification at that scale. R3A pairs multi-agent fault localization with a search that keeps only verified patches [28], Clover dispatches repair operations across agents and symbolic solvers under the same search [29], and MechMem-RTL reuses verifier-confirmed repair records whose stored trigger matches the new failure [30]. VerilogCoder adds AST-based waveform tracing to graph planning against golden testbenches [6], VeriTrace queries signals and time windows [8], and FVDebug works from formal counterexamples and specifications [7]. BluesFL instead stops at localization, reading internal signals along co-simulation-guided slices of a 19K-line design [9]. HWE-Bench moves to the repository level, giving an agent a problem statement and the whole repository and judging its fix by project-native flows, so it contributes a benchmark rather than a method [15]. Our work targets this setting, where the faulty module and a self-contained reproduction are not given, and repairs may cross file and module boundaries.

### B. Repository-Level Coding Agents

Repository-level coding systems retrieve, call tools, and validate to address issue reports, and SWE-agent designs the interface that carries them [31]. Agentless fixes a pipeline of localization, repair, and validation [32]; graph-guided agents explore adaptively, LocAgent over a heterogeneous entity graph [33], GraphLocator over causal links from sub-issues to code entities [34], and repository memory by adding commits, issues, and summaries [35]. These systems target software code repositories; hardware repositories instead rely on knowledge graphs over Verilog sources and their descriptions [36]. Our work moves this setting to end-to-end repair of existing RTL repositories, where reproduction, localization, and validation must be established in project-specific flows.

## III. METHODOLOGY

### A. Overview

Figure 2 shows the evidence-guided loop. The method centers on one shared artifact, the waveform of a reproduced failing run. Failure grounding first turns the problem statement into a concrete failing execution and its waveform. Waveform-guided localization then gives the agent a reusable localization toolbox for extracting, tracing, and semantically interpreting signal behavior from that execution. Consistency-aware repair

and validation finally turn a local diagnosis into coordinated repository edits and check whether the revised design removes the same violation while preserving the reproduced scenario. Section III-B details failure grounding, Section III-C presents waveform-guided localization, and Section III-D describes consistency-aware repair and validation.

### B. Failure Grounding

*a) Failure specification extraction.* The held-out fail-to-pass tests are not visible to the agent, so the framework must first construct its own failing run. To keep that run tied to the problem statement, the framework parses it into three checkable elements: the preconditions  $P$  under which the bug should appear, the triggering event or transaction  $E$ , and the required behavior  $Q$  that is violated.

*b) Grounded reproduction.* A reproduced run is useful only if later localization reasons about the same failure mechanism described in the problem statement. The framework therefore constructs one failing execution for the reported scenario and enables tracing on the unmodified repository. The resulting run passes an alignment check and is accepted as grounding evidence only when the waveform shows that  $P$  and  $E$  actually occur and that  $Q$  is observably violated. The framework also requires the stimulus and configuration to stay within the repository’s intended usage assumptions, such as legal parameter settings, interface protocols, and supported operating modes. Unclear assumptions are recorded explicitly instead of being promoted to facts.

*c) Failure anchor formulation.* Later modules need one stable representation of the reproduced failure; otherwise localization and validation may drift to different scenarios. Failure grounding therefore outputs a failure anchor  $F = (S, V, R)$  with three elements:

- $S = (P, E)$  is the reproduced situation, where  $P$  collects the preconditions under which the bug should appear and  $E$  denotes the triggering event or transaction.
- $V = (Q, \ell, t)$  is the observed violation, where  $Q$  is the required behavior, contradicted at location  $\ell$  and cycle or interval  $t$ .
- $R$  stores the stimulus and configuration used to reproduce the failure.

Localization later consumes  $(V, R)$  to explain the observed symptom, while validation replays  $R$  and checks that  $S$  still occurs after a candidate repair and that  $V$  disappears. The anchor keeps later diagnosis and validation aligned to the same reproduced failure.

### C. Waveform-Guided Localization

Given the failure anchor  $F = (S, V, R)$ , localization takes the observed violation  $V$  in reproduced run  $R$  as input and returns  $L = (M, T)$ , where  $M$  is a candidate mechanism and  $T$  is an evidence trail. The challenge is that the waveform is large while only a small part is causally relevant. The framework therefore provides a reusable localization toolbox that makes the failing execution systematically queryable together

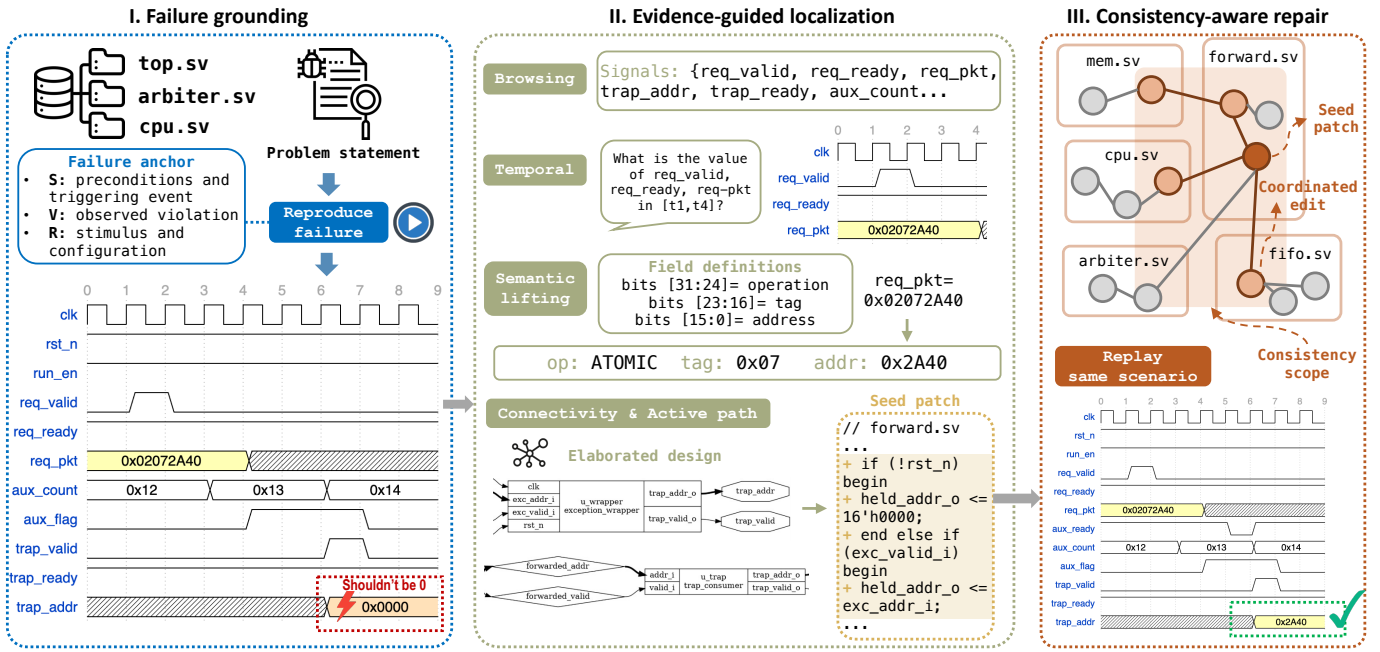


Fig. 2. The framework runs one evidence-guided loop. It reconstructs a failing run and dumps its waveform, helps the agent localize a candidate mechanism with the localization toolbox, and then checks whether a coordinated patch preserves the scenario while removing the observed violation.

TABLE I

Query classes the localization toolbox provides, their role, and their returned artifacts.

| Query class      | Localization role                                                | Artifact                                                |
|------------------|------------------------------------------------------------------|---------------------------------------------------------|
| Browsing         | Align problem-statement or source names with waveform hierarchy. | Scope/signal candidates.                                |
| Temporal         | Bound the violation window.                                      | Divergence windows and timelines.                       |
| Connectivity     | Trace from symptom signals to producing logic.                   | Driver/load/fan-in links.                               |
| Active path      | Recover the executed dependency chain.                           | Executed driver chain.                                  |
| Semantic lifting | Map raw values to RTL-level semantics.                           | Decoded fields, symbolic states, and aligned waveforms. |

with the elaborated design under the reproduced configuration. This setup supports queries across module boundaries, instantiations, and generated hierarchy.

a) *Waveform query interface.* Raw waveforms are too large and heterogeneous for direct free-form inspection, so localization needs a small set of structured queries that progressively narrow the search space. Localization therefore uses five query classes, summarized in Table I. Together, they move the analysis from a symptom signal, to a bounded interval, to the exercised logic, and finally to design-level semantics.

Beyond structural queries, localization also relies on semantic lifting to turn raw waveform values into design-level concepts. This includes decoding packed fields, mapping state encodings to symbolic states, and aligning related waveforms on a shared timeline. These lifted artifacts let the agent reason

about candidate mechanisms at the RTL semantic level rather than at the level of raw integers.

b) *Evidence-narrowing loop.* Even with a query interface, localization remains ambiguous unless evidence is used to eliminate competing explanations step by step. Localization therefore applies these primitives as a loop: it time-bounds the violation in  $V$ , traces backward over active drivers, invokes semantic lifting when raw values are not yet interpretable, and widens the waveform window or compares alternatives when the current evidence is insufficient. It terminates only when it can summarize a candidate mechanism  $M$  together with an evidence trail  $T$  that records both supporting observations and remaining ambiguities. Because each narrowing step is backed by values observed in  $R$ , the final mechanism remains linked to an explicit evidence trail.

#### D. Consistency-Aware Repair and Validation

Consistency-aware repair addresses the second repository-level failure mode: a correct local diagnosis does not yet imply a repository-consistent fix. Once one site changes, the same mechanism may still be represented in connected definitions, bindings, or replicated realizations, so the fix must be propagated and rechecked under the same reproduced scenario. Given  $F$  and localization output  $L = (M, T)$ , the framework builds a coordinated patch  $C$ , expands it across structurally linked locations, and validates that the original scenario is preserved while the reported violation is removed.

a) *Repository relation graph.* To determine where a seed repair must propagate, the framework constructs a repository relation graph over repair sites. Nodes denote repair sites at the declaration, implementation, structural-binding, and

TABLE II

Instantiation sites the source declares against the instances the configured design contains, for five modules taken from the cases we elaborated.

| Repository | Module                   | source | elaborated |
|------------|--------------------------|--------|------------|
| Ibex       | ibex_csr                 | 21     | 15         |
| Ibex       | ibex_counter             | 3      | 12         |
| CVA6       | lzc                      | 39     | 115        |
| CVA6       | spill_register_flushable | 1      | 60         |
| OpenTitan  | prim_subreg              | 104    | 104        |

generated-RTL levels, together with their enclosing modules or files when needed for propagation. Edges encode structural relations that may require coordinated updates across these sites.

The graph combines source-level and elaboration-level evidence. Source parsing recovers symbol relations, structural containment, and local dependency information. HDL elaboration recovers the resolved hierarchy and concrete binding contexts, including instantiated connectivity, parameter resolution, width resolution, and generated replicas, thereby turning source templates into concrete repair contexts. For generated RTL, the framework links emitted RTL locations back to the generator source or shared template that produced them, using source maps, line directives, or generator metadata when available. When such information is unavailable, the framework adds conservative structurally matched links and marks them as uncertain. Starting from the sites edited by the seed patch, closure over  $G_R$  yields the candidate propagation scope.

Table II shows why the relation graph follows the elaborated design: counting textual instantiation sites overstates the scope when a site is never built, as for *ibex\_csr* behind a disabled PMP gate, and understates it when one site expands into many, as for CVA6’s *spill\_register\_flushable*. A closure over source-level relations therefore demands edits at places this configuration never instantiates and misses replicated instances it does, so the scope it returns is not the scope the design has, and a seed repair propagated over it is incomplete in exactly the way repository-level bugs are. For generated RTL the graph also links emitted RTL back to its generator, and OpenTitan’s counts are a lower bound: no Verilator model can be built for it, so the graph is built from a partial pyslang netlist without parameter values.

*b) Consistency closure and validation.* Algorithm 1 summarizes this loop in two phases. In Phase I, the agent uses  $G_R$  to enumerate the sites that are structurally coupled to its edited locations. This traversal intentionally over-approximates the consistency scope, so the agent next inspects the reached sites and marks the subset that truly requires synchronized changes. In Phase II, the resulting coordinated patch is validated by replaying the reproduction recipe  $R$  from  $F$ . A patch is accepted only if the replay still realizes the reproduced preconditions  $P$  and trigger  $E$ , and the required behavior  $Q$  is no longer violated. If replay breaks  $P$  or  $E$ , then the current patch has changed the reproduced operating condition and the

### Algorithm 1 Consistency-aware repair and validation

**Require:**  $F = (S, V, R)$ ,  $L = (M, T)$ ,  $G_R$

**Ensure:** validated patch  $C$

```

1:  $(P, E) \leftarrow S$   $\triangleright P$ : reproduced preconditions;  $E$ : trigger
2:  $(Q, \ell, t) \leftarrow V$   $\triangleright Q$ : required behavior
3:  $C \leftarrow \text{SEEDPATCH}(L)$ 
4: repeat
  Phase I: Consistency-scope construction
5:  $E_C \leftarrow \text{EDITEDSITES}(C)$ 
6:  $U \leftarrow \text{CLOSURE}(E_C, G_R)$ 
7:  $U_{\text{edit}} \leftarrow \text{SELECTREQUIRED}(U, C, T)$ 
8:  $C \leftarrow \text{PROPAGATEREPAIR}(C, U_{\text{edit}})$ 
  Phase II: Replay-based validation
9:  $O \leftarrow \text{REPLAY}(R, C)$ 
10: if  $\neg \text{PRESERVE SCENARIO}(O, P, E)$  then
11:    $C \leftarrow \text{REFINEPATCH}(C, O, L)$ 
12: else if  $\neg \text{ELIMINATEVIOLATION}(O, Q)$  then
13:    $U \leftarrow \text{EXPANDSCOPE}(U, G_R, O)$ 
14:    $U_{\text{edit}} \leftarrow \text{SELECTREQUIRED}(U, C, T)$ 
15:    $C \leftarrow \text{PROPAGATEREPAIR}(C, U_{\text{edit}})$ 
16: end if
17: until  $\text{PRESERVE SCENARIO}(O, P, E) \wedge \text{ELIMINATEVIOLATION}(O, Q)$ 
18: return  $C$ 

```

seed repair itself must be refined. If  $P$  and  $E$  are preserved but  $Q$  is still violated, then the local edit remains insufficient and the consistency scope should be expanded before the next attempt.

## IV. EXPERIMENTS

### A. Experiment Setup

*a) Benchmark and metric.* We evaluate on HWE-Bench [15], which collects 417 real bug-fixing tasks from six open-source RTL repositories. Each task provides a repository at a buggy revision and a problem statement, and success is judged by the repository’s own held-out fail-to-pass tests. We report the resolved rate, the share of tasks whose held-out fail-to-pass tests all pass. The tasks cover RISC-V cores, SoCs, processors, and security roots of trust. OpenTitan needs a commercial simulator (VCS) while the other five repositories run under Verilator, and 29 of OpenTitan’s 245 tasks cannot be adapted to a runnable VCS environment in our setup. Those 29 are skipped for every condition, so every result below is over the remaining 388 tasks, the agent never sees the held-out tests, and all denominators use the same pool.

*b) Implementation.* For the baseline we run each model under its own scaffold, unchanged: DeepSeek V4 Flash (0731) under OpenHands SDK, DeepSeek V4 Pro under Claude Code, and Kimi K2.6 under Kimi CLI.

Relative to the baseline, the framework adds three modules. First, failure grounding reproduces a failing run and stores it as an FST waveform. Second, waveform-guided localization exposes a localization toolbox whose scripts browse the waveform hierarchy, bound the violation in time, trace connectivity and the active path, and lift raw values into RTL-level semantics. Third, consistency-aware repair and validation propagate a local patch to structurally coupled sites and then replay the reproduced scenario to check the result.

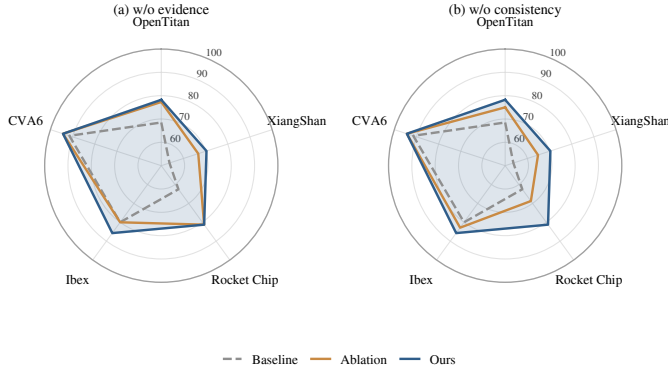


Fig. 3. Resolved rate per repository, DeepSeek V4 Flash (0731), under the baseline, the framework, and the framework with one arm ablated. The radial axis is percent and starts at 50%.

## B. Results and Analysis

We read the experiments around four questions and close with what the framework costs to run.

**RQ1:** Does the framework resolve more tasks?

**RQ2:** How much does each arm contribute?

**RQ3:** What changes in how the agent works?

**RQ4:** Why do the remaining failures persist?

*RQ1: Does the framework resolve more tasks?* Table III shows the main result: every model improves on its baseline, and the gain is spread over every repository where the baseline leaves headroom, concentrating on the two largest. OpenTitan is a hardware security root of trust whose deep module hierarchy and generated register layers spread one behavioral change over many files; XiangShan is an out-of-order RISC-V core written in Chisel, so a fix usually has to reach many parameterized instances. Both carry the most headroom, and most of their missed tasks need edits in several files, the coordinated setting the consistency module is built for. Across the pool every model improves on its own scaffold, so the gain is the framework’s rather than a stronger agent’s. These failures need a mechanism and a complete patch, and the framework supplies both, from a reproduced run and the elaborated scope.

*RQ2: How much does each arm contribute?* Figure 3 reads the pool three ways per repository: the baseline, the framework, and the framework with one arm ablated. The evidence arm drops the alignment check and the localization toolbox; the consistency arm drops scope construction and replay validation. Either ablation costs the framework repairs, so neither arm is redundant. The evidence arm works from a single recorded execution, so it pays off only where the failure is visible in that waveform. The consistency arm reads the repository and the elaborated hierarchy, so it catches the coupled edits no waveform shows and loses repairs on Rocket Chip, where the evidence arm loses none. Between the two, the consistency arm matters most: the repairs it recovers need edits at coupled modules, and locating those modules is itself a form of localization that a single waveform cannot supply. The two arms therefore cover different halves of the job, evidence

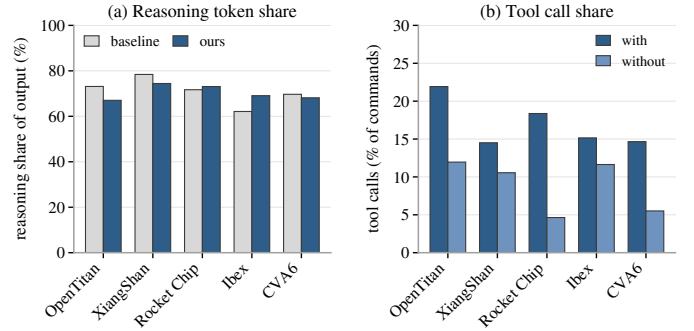


Fig. 4. Agent behavior per repository with DeepSeek V4 Flash (0731): reasoning token share under the framework and the baseline (a), and tool call share with and without the localization toolbox (b).

supplying the mechanism and consistency deciding whether the patch built on it reaches every module the fix requires.

*RQ3: What changes in how the agent works?*

*a) Reasoning share.* Reading a waveform does not displace the agent’s reasoning. Figure 4a compares the share of a run’s output that is reasoning text. The median share falls on OpenTitan and XiangShan, rises on Rocket Chip and Ibex, and barely moves on CVA6; every shift stays inside the spread the baseline shows across its own cases. With the toolbox in hand the agent therefore stays a reasoner rather than an executor and combines reasoning with execution, which is what turns the evidence into repairs.

*b) Tool call share.* Figure 4b reports the share of the agent’s commands that are tool calls, and with the toolbox mounted that share is higher than in the arm without it, so the agent reaches for it on its own. The arm without it calls tools too, but those are scripts the agent writes itself, and nothing guarantees that they are correct. Either the agent notices a broken script and rewrites it, which wastes tokens and puts queries too complex to write by hand out of reach, or the script is syntactically sound but semantically wrong, and the values it returns mislead the repair. Under the framework the toolbox scripts are verified, so a call returns evidence the agent can act on and its tool use is efficient.

*RQ4: Why do the remaining failures persist?* Table IV splits the failures the baseline leaves unresolved per repository by the evidence available for them: the share the alignment check accepts, the share it rejects, and the share with no waveform at all. The *none* share is where the repository’s own simulation flow cannot be driven in our setup, and the *rejected* share where a waveform exists but is too sparse, or holds a level or subtree that does not carry the failure. They persist for two reasons.

*a) Where the evidence does not help.* These failures need information no run of the buggy revision carries, either in the interface the task defines or in the RTL generator. On *Ibex-104* the held-out test compiles against three CSR ports that revision does not contain, so the repair must invent an interface whose names live only in the test and in the golden patch, the repository’s own fix that the agent never sees. A waveform

TABLE III  
Resolved tasks per repository; each cell is *resolved/tasks*.

| Model                         | Scaffold      | OpenTitan | Ibex  | CVA6  | Caliptra | XiangShan | Rocket Chip | Total   |
|-------------------------------|---------------|-----------|-------|-------|----------|-----------|-------------|---------|
| <i>Baseline</i>               |               |           |       |       |          |           |             |         |
| DeepSeek V4 Flash (0731)      | OpenHands SDK | 148/216   | 28/35 | 32/35 | 16/16    | 29/54     | 20/32       | 273/388 |
| DeepSeek V4 Pro               | Claude Code   | 132/216   | 20/35 | 30/35 | 10/16    | 32/54     | 19/32       | 243/388 |
| Kimi K2.6                     | Kimi CLI      | 130/216   | 28/35 | 30/35 | 13/16    | 34/54     | 23/32       | 258/388 |
| <i>Evidence-guided (ours)</i> |               |           |       |       |          |           |             |         |
| DeepSeek V4 Flash (0731)      | OpenHands SDK | 169/216   | 30/35 | 33/35 | 16/16    | 38/54     | 26/32       | 312/388 |
| DeepSeek V4 Pro               | Claude Code   | 157/216   | 28/35 | 32/35 | 16/16    | 40/54     | 24/32       | 297/388 |
| Kimi K2.6                     | Kimi CLI      | 160/216   | 31/35 | 33/35 | 16/16    | 37/54     | 26/32       | 303/388 |

TABLE IV  
Failures the DeepSeek V4 Flash (0731) baseline leaves unresolved, by the evidence available. The three columns partition a repository’s failures and are given in percent.

| Repository  | accepted | rejected | none |
|-------------|----------|----------|------|
| OpenTitan   | 58.8     | 4.4      | 36.8 |
| XiangShan   | 56.0     | 24.0     | 20.0 |
| Rocket Chip | 41.7     | 8.3      | 50.0 |
| Ibex        | 57.1     | 14.3     | 28.6 |
| CVA6        | 66.7     | 0.0      | 33.3 |
| Total       | 56.5     | 9.6      | 33.9 |

cannot name a port that was never built. On *RocketChip-1493* the waveform shows the asynchronous crossing behind the stalled boot-ROM access and the agent reroutes the boot ROM in one Scala file, while the golden patch introduces a control bus and migrates the peripherals onto it, a generator-level decision that no value in the waveform ranks above the smaller edit.

b) *Where the agent does not use it well.* Where a waveform existed, the agent usually consulted it and still failed. That step needs the protocol or micro-architectural contract behind the signals, which the agent must supply from hardware knowledge, and where that knowledge is incomplete the repair follows the values to the wrong edit. On *XiangShan-1242* the queries return the comparison that stalls a load, and the agent reads the empty store-set pointer as a missing assignment instead of seeing that the pointer is defined on the ROB index, which the golden patch rewrites eleven Scala files to carry. On *OpenTitan-22878* it reassembles the key-manager payload in the file the problem statement implicates but keeps the original field order and leaves the valid-side and debug-invalid mappings alone, because the values show which fields are wrong and nothing about the order and handshake the protocol fixes.

*Cost.* Figure 5 draws the two curves per repository, and they stay close on all four. The framework adds a thin band on OpenTitan and XiangShan, less on Rocket Chip, and on Ibex the two are hard to tell apart, so no repository pays much for the evidence. The waveform is written once and read through scripts, which the agent never loads as a file,

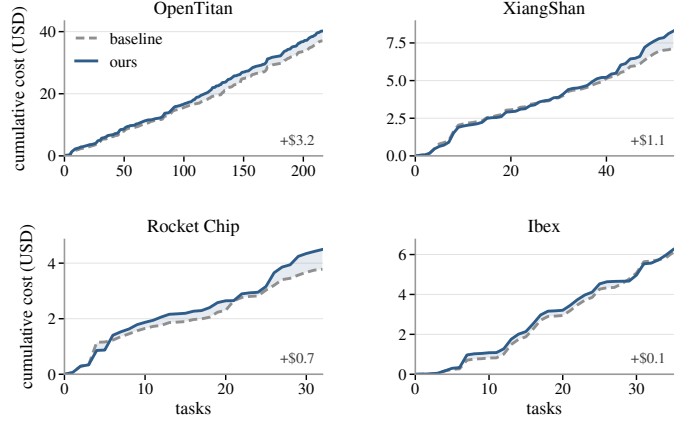


Fig. 5. Cumulative model spend per repository with DeepSeek V4 Flash (0731): the baseline and the framework, with the shaded band showing the marginal spend the framework adds.

so a query returns the few values it asks for and a waveform of tens of thousands of signals costs the tokens of one short command. The toolbox also cuts what the baseline spends on guessing, since the waveform lets the agent test a hypothesis instead of reading files on suspicion, and those passes offset what the queries add. OpenTitan grows most and is the most complex, with the largest waveforms.

## V. CONCLUSION

Repository-level RTL repair must localize failures across files, modules, and clock cycles and then propagate fixes consistently, without a specification or reference design to certify either step. We presented an evidence-guided framework whose three modules ground the repair in a reproduced failing run, narrow the observed violation into a candidate mechanism and an evidence trail, and expand that seed into a coordinated, replay-checked patch. On the 388 HWE-Bench tasks we evaluate, the framework resolves more tasks than the baseline for every model we test. Trajectories show agents extracting signal values, lifting them into design semantics, and using reruns to expose incomplete repairs, while remaining failures highlight the gap between obtaining relevant evidence and producing complete repository-level fixes.

## REFERENCES

- [1] H. D. Foster, "Trends in functional verification," in *Proc. Design Automation Conf. (DAC)*, 2015, pp. 1–6.
- [2] X. Xin, J. Lou, J. Li, J. Yan, P. Xiao, D. Wu, H. Li, W. Lu, W. Fan, X. Qu, Y. Zhao, M. Yu, Z. Di, and Y. Lin, "GoGoTB: Agentic RTL verification with specification-grounded coverage closure," arXiv:2607.26181, 2026.
- [3] B. Yao, N. Wang, J. Zhou, X. Wang, H. Gao, Z. Jiang, and N. Guan, "Location is key: Leveraging large language model for functional bug localization in Verilog," arXiv:2409.15186, 2024.
- [4] K. Xu, J. Sun, Y. Hu, X. Fang, W. Shan, X. Wang, and Z. Jiang, "MEIC: Re-thinking RTL debug automation using LLMs," arXiv:2405.06840, 2024.
- [5] Y. Wang, C. Liu, J. Zhang, L. Zhang, L. Cheng, X. Li, and H. Li, "VeriPilot: An LLM-powered Verilog debugging framework," arXiv:2606.23759, 2026.
- [6] C.-T. Ho, H. Ren, and B. Khailany, "VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool," in *Proc. AAAI Conf. on Artificial Intelligence (AAAI)*, 2025, pp. 300–307.
- [7] Y. Bai, G. B. Hamad, C.-T. Ho, S. Suhaib, and H. Ren, "FVDebug: An LLM-driven debugging assistant for automated root cause analysis of formal verification failures," arXiv:2510.15906, 2025.
- [8] Y.-T. Liu and C. Yu, "VeriTrace: Human-like temporal exploration completes agentic action space," ICLAD; arXiv:2608.02878, 2026.
- [9] Z. Liu, X. Mao, D. Yang, J. He, Y. Qin, G. Zhang, Y. Lei, J. Xu, and J. Wu, "Debug like a human: Scaling LLM-based fault localization to processor design via block-level instruction-oriented slicing," arXiv:2605.17290, 2026.
- [10] W. Fu, K. Yang, R. G. Dutta, X. Guo, and G. Qu, "LLM4SecHW: Leveraging domain specific large language model for hardware debugging," arXiv:2401.16448, 2024.
- [11] H. Ahmad, Y. Huang, and W. Weimer, "CirFix: Automatically repairing defects in hardware design code," in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022, pp. 990–1003.
- [12] K. Laeuffer, B. Fajardo, A. Ahuja, V. Iyer, B. Nikolic, and K. Sen, "RTL-Repair: Fast symbolic repair of hardware design code," in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 867–881.
- [13] G. Stracquadanio, S. Medya, S. Quer, and D. Pal, "VeriBug: An attention-based framework for bug-localization in hardware designs," arXiv:2401.09494, 2024.
- [14] X. Meng, K. Feng, X. Zhao, Z. Zhang, Y. Lei, X. Mao, and J. Wu, "VeriBugBench: An empirically grounded framework for constructing Verilog RTL debugging benchmarks," arXiv:2609.18022, 2026.
- [15] F. Cui, H. Hou, Z. Luo, C. Yin, and Y. Liang, "HWE-Bench: Benchmarking LLM agents on real-world hardware bug repair tasks," arXiv:2604.14709, 2026.
- [16] J. de Kleer and B. C. Williams, "Diagnosing multiple faults," *Artificial Intelligence*, vol. 32, no. 1, pp. 97–130, 1987.
- [17] R. Reiter, "A theory of diagnosis from first principles," *Artificial Intelligence*, vol. 32, no. 1, pp. 57–95, 1987.
- [18] A. Feldman and G. Provan, "A model-based active testing approach to sequential diagnosis," arXiv:1401.3850, 2014.
- [19] P. Rodler, "Towards optimizing Reiter's HS-tree for sequential diagnosis," arXiv:1907.12130, 2019.
- [20] H. Chockler and J. Y. Halpern, "Responsibility and blame: A structural-model approach," *Journal of Artificial Intelligence Research*, vol. 22, pp. 93–115, 2004.
- [21] J. Y. Halpern and J. Pearl, "Causes and explanations: A structural-model approach. Part I: Causes," *The British Journal for the Philosophy of Science*, vol. 56, no. 4, pp. 843–887, 2005.
- [22] J. Townsend, C. Pravin, K. H. Ngan, and M. Parizy, "A study on the impact of fault localization granularity for repository-scale code repair tasks," arXiv:2604.00167, 2026.
- [23] P. Zhao, B. Yang, B. Le, F. Liu, and H. Tian, "Beyond localization: Recoverable headroom and residual frontier in repository-level RAG-APR," arXiv:2603.29067, 2026.
- [24] S. Vasudevan, D. Sheridan, S. Patel, D. Tcheng, B. Tuohy, and D. Johnson, "GoldMine: Automatic assertion generation using data mining and static analysis," in *Proc. Design, Automation and Test in Europe (DATE)*, 2010, pp. 626–629.
- [25] S. Hertz, D. Sheridan, and S. Vasudevan, "Mining hardware assertions with guidance from static analysis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 6, pp. 952–965, 2013.
- [26] V. Pulavarthi, D. Nandal, S. Dan, and D. Pal, "AssertionBench: A benchmark to evaluate large-language models for assertion generation," arXiv:2406.18627, 2024.
- [27] Y. Liu, D. Cummings, and N. Vadlamudi, "WaveformQA: Benchmarking LLM temporal reasoning on digital waveforms," ICLAD; arXiv:2607.20638, 2026.
- [28] Z. Luo, F. Cui, K. Zhou, R. Guo, M. Xia, H. Hou, and Y. Liang, "R3A: Reliable RTL repair framework with multi-agent fault localization and stochastic tree-of-thoughts patch generation," arXiv:2511.20090, 2025.
- [29] Z. Luo, Y. Xu, R. Guo, F. Cui, K. Zhou, M. Xia, H. Hou, Y. Luo, and Y. Liang, "Clover: A neural-symbolic agentic harness with stochastic tree-of-thoughts for verified RTL repair," arXiv:2604.17288, 2026.
- [30] M. Cheng, J. Gao, J. Cui, and K. Zhong, "MechMem-RTL: Reusing verified mechanism memories for LLM-based RTL repair," arXiv:2607.17053, 2026.
- [31] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," NeurIPS; arXiv:2405.15793, 2024.
- [32] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024.
- [33] Z. Chen, X. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang, "LocAgent: Graph-guided LLM agents for code localization," arXiv:2503.09089, 2025.
- [34] W. Liu, C. Peng, P. Gao, A. Liu, W. Zhang, H. Zhao, and Z. Jin, "GraphLocator: Graph-guided causal reasoning for issue localization," arXiv:2512.22469, 2025.
- [35] B. Wang, W. Xu, Y. Li, M. Gao, Y. Xie, H. Sun, and D. Chen, "Improving code localization with repository memory," arXiv:2510.01003, 2025.
- [36] A. Szeskin, I. Lieder, D. Tummalapalli, and A. Armon, "Retrieval on Verilog repositories: A knowledge-graph based solution," in *NeurIPS Workshop on Machine Learning for Systems*, 2025.